INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
CAMPUS BOA VIAGEM
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

Halef de Morais

Feira Digital do Sertão

Sistema web orientado a objetos em PHP

Relatório do trabalho individual da disciplina Programação Web I
Professor: Prof. Dr. Renato William Rodrigues de Souza

Boa Viagem, CE
2026

Sumário

1 Introdução **2**

2 Requisitos **2**
2.1 Funcionalidades 2
2.2 Regras de negócio 2

3 Modelagem **3**
3.1 Herança 3
3.2 Composição 3
3.3 Encapsulamento 4

4 Implementação **4**
4.1 Encapsulamento e construtores (OO1 e OO4) 4
4.2 Herança e sobrescrita (OO2) 4
4.3 Polimorfismo (OO3) 5
4.4 Regras de negócio com exceções (OO5) 5
4.5 Composição (OO6) 6
4.6 Organização em arquivos e conversão para JSON (OO7 e OO8) 6

5 Camada web **6**
5.1 Rotas 7
5.2 Páginas e status 404 (W1) 7
5.3 Rota JSON (W2) 7
5.4 Formulário com POST (W3) 8

6 Testes **8**

7 Ferramentas utilizadas **10**

8 Uso de LLM **10**

9 Dificuldades, aprendizados e conclusão **11**

1 Introdução

O tema que escolhi foi a Feira Digital do Sertão. Nas feiras do sertão, cada produto é vendido de um jeito: o pão de queijo por unidade, a carne de sol por peso e com compra mínima, e o iogurte e o queijo fresco com prazo de validade curto. O sistema organiza o catálogo da feira e os pedidos dos clientes aplicando essas regras. Ele serve a feirantes e organizadores que querem montar pedidos sem erro de quantidade, de preço ou de validade.

O sistema foi feito em PHP, todo organizado em classes, e junta o que vimos nas Aulas 01 a 03: HTTP, a requisição e a resposta no PHP, e orientação a objetos. A Seção 2 traz os requisitos e as regras de negócio. A Seção 3 mostra a modelagem. A Seção 4 mostra onde cada conceito de orientação a objetos aparece no código. A Seção 5 explica a camada web. A Seção 6 traz os testes. As Seções 7 e 8 falam das ferramentas e do uso de LLM, e a Seção 9 traz as dificuldades e a conclusão.

2 Requisitos

2.1 Funcionalidades

- **Páginas** escolhidas pela query string (`?pagina=`), todas com o mesmo cabeçalho, menu e rodapé: início, lista de produtos, detalhe de um produto, lista de pedidos e detalhe de um pedido.
- **Status 404** para página inexistente, produto inexistente e pedido inexistente.
- **Formulário de novo pedido** (método POST), com o nome do cliente e até três produtos com suas quantidades. Se uma regra recusar os dados, a mensagem da exceção aparece na página.
- **Rota JSON** (`?formato=json`), que funciona sozinha ou junto de `pagina` e `id`.
- **Catálogo de exemplo** com 6 produtos e 2 pedidos, criados no próprio `index.php`.

2.2 Regras de negócio

Todas as regras são recusadas com `throw new Exception(...)`. O PDF pede pelo menos 3, e o sistema tem 10. Além delas, o produto perecível tem uma regra de cálculo: 30% de desconto quando faltam 2 dias ou menos para vencer.

Nº	Regra	Onde está (classe e método)
1	Produto precisa de nome	`Produto::__construct()`
2	Preço maior que zero	`Produto::__construct()`
3	Quantidade maior que zero	`Produto::validarQuantidade()`
4	Produto unitário só aceita quantidade inteira	`ProdutoUnitario::validarQuantidade()`
5	Produto por peso exige a compra mínima	`ProdutoPorPeso::validarQuantidade()`
6	Peso mínimo maior que zero	`ProdutoPorPeso::__construct()`
7	Validade em data válida (AAAA-MM-DD)	`ProdutoPerecivel::__construct()`
8	Produto perecível não pode estar vencido	`ProdutoPerecivel::__construct()`
9	Pedido precisa de cliente	`Pedido::__construct()`
10	Produto não pode se repetir no pedido	`Pedido::adicionarItem()`

O formulário ainda recusa pedido sem nenhum produto, produto fora da lista e quantidade que não seja número.

3 Modelagem

A Figura 1 mostra o diagrama de classes, que também está em `docs/modelagem.md`. O sistema tem 6 classes: `Produto` (mãe), `ProdutoUnitario`, `ProdutoPorPeso` e `ProdutoPerecivel` (filhas), `ItemPedido` e `Pedido`.

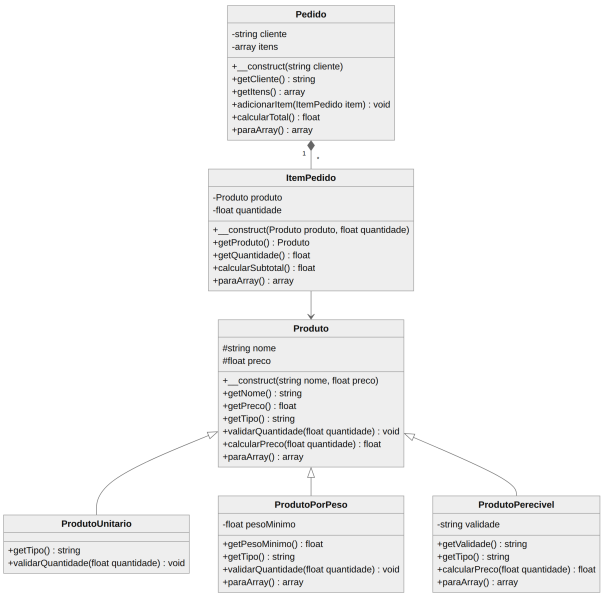


Figura 1: Diagrama de classes do sistema

3.1 Herança

A classe mãe é `Produto`, que guarda o nome e o preço, dados que todo produto da feira tem. A frase "X é um Y" vale para as três filhas: um produto unitário é um produto, um produto por peso é um produto e um produto perecível é um produto. Cada filha chama `parent::__construct()` para reaproveitar a validação do nome e do preço, e sobrescreve o que muda no seu tipo de venda:

- `getTipo()` é sobrescrito nas três filhas.
- `validarQuantidade()` é sobrescrito em `ProdutoUnitario` (só aceita número inteiro) e em `ProdutoPorPeso` (exige a compra mínima). As duas chamam `parent::validarQuantidade()` antes, para manter a regra de quantidade maior que zero.
- `calcularPreco()` é sobrescrito em `ProdutoPerecivel`, que aplica o desconto perto do vencimento.
- `paraArray()` é sobrescrito em `ProdutoPorPeso` e `ProdutoPerecivel`, para incluir os dados extras no JSON.

3.2 Composição

`Pedido` guarda uma lista de `ItemPedido`, e cada `ItemPedido` guarda um `Produto` e uma quantidade. A relação é "tem um": um pedido tem itens, e não é um item. O item só existe dentro de um pedido, e por isso a ligação é uma composição. Já a ligação entre `ItemPedido` e `Produto`

é uma seta simples, porque o produto existe sozinho no catálogo e vários itens podem apontar para ele. O `Pedido` tem o método `adicionarItem()` para montar a lista e `calcularTotal()` para somar os subtotais.

3.3 Encapsulamento

Nenhum atributo é `public`. Em `Produto`, `nome` e `preco` são `protected`, para as filhas usarem. Todos os outros são `private`. Não criei setters: os dados são validados uma vez no construtor e depois não mudam, então não há como deixar um objeto em estado inválido.

4 Implementação

Todos os arquivos de `src/` começam com `declare(strict_types=1);`. Nos trechos abaixo mostro só o essencial de cada requisito.

4.1 Encapsulamento e construtores (OO1 e OO4)

O construtor de `Produto` recebe os dados tipados, valida e só depois guarda nos atributos `protected`. O objeto nunca chega a existir com dados inválidos.

```
class Produto
{
    protected string $nome;
    protected float $preco;

    public function __construct(string $nome, float $preco)
    {
        if (trim($nome) === '') {
            throw new Exception('O nome do produto não pode ficar vazio.');
        }
        if ($preco <= 0) {
            throw new Exception('O preço do produto deve ser maior que zero.');
        }

        $this->nome = trim($nome);
        $this->preco = $preco;
    }

    public function getNome(): string
    {
        return $this->nome;
    }
}
```

4.2 Herança e sobrescrita (OO2)

A classe `ProdutoPorPeso` usa `extends`, chama o construtor da mãe com `parent::` e sobrescreve `validarQuantidade()`. O atributo `pesoMinimo` é `private`, porque é uma regra só desta classe.

```
class ProdutoPorPeso extends Produto
{
    private float $pesoMinimo;

    public function __construct(string $nome, float $preco, float $pesoMinimo)
```

```
    {
        parent::__construct($nome, $preco);
        if ($pesoMinimo <= 0) {
            throw new Exception('O peso mínimo deve ser maior que zero.');
        }
        $this->pesoMinimo = $pesoMinimo;
    }

    public function validarQuantidade(float $quantidade): void
    {
        parent::validarQuantidade($quantidade);

        if ($quantidade < $this->pesoMinimo) {
            $minimo = str_replace('.', ',', (string) $this->pesoMinimo);
            throw new Exception("A compra mínima de {$this->nome} é {$minimo} kg.");
        }
    }
}
```

4.3 Polimorfismo (OO3)

No `teste.php`, um `foreach` percorre objetos de classes diferentes e chama os mesmos métodos, `getTipo()` e `calcularPreco()`. Cada objeto responde do seu jeito: o iogurte, que vence em 1 dia, sai com desconto, e os outros não.

```
$produtos = [$pao, $carne, $iogurte, $queijo];

foreach ($produtos as $produto) {
    echo "{$produto->getNome()} [{$produto->getTipo()}]: 2 x "
        . formatar($produto->getPreco())
        . ' = ' . formatar($produto->calcularPreco(2)) . "\n";
}
```

4.4 Regras de negócio com exceções (OO5)

A regra nasce com `throw new Exception(...)` dentro da classe e é tratada por quem usa o objeto, com `try/catch`. Abaixo, a regra do produto repetido (em `Pedido`) e o `try/catch` que a trata no `teste.php`. No `index.php`, um `try/catch` parecido mostra a mensagem na página do formulário.

```
// src/Pedido.php
foreach ($this->itens as $existente) {
    if (strcasecmp($existente->getProduto()->getNome(), $nome) === 0) {
        throw new Exception("O produto {$nome} já está no pedido.");
    }
}

// teste.php
try {
    $pedido->adicionarItem(new ItemPedido($pao, 1));
    echo "ERRO: o produto repetido foi aceito\n";
} catch (Exception $e) {
    echo "OK, recusado: {$e->getMessage()}\n";
}
```

4.5 Composição (OO6)

`Pedido` guarda a lista `$itens`, tem um método para adicionar e outro que calcula o total sobre a lista. O subtotal de cada item vem do `calcularPreco()` do produto, então o desconto do perecível entra no total sem o `Pedido` precisar saber disso.

```
private array $itens = [];

public function adicionarItem(ItemPedido $item): void
{
    // ... recusa produto repetido (trecho anterior)
    $this->itens[] = $item;
}

public function calcularTotal(): float
{
    $total = 0.0;

    foreach ($this->itens as $item) {
        $total += $item->calcularSubtotal();
    }

    return round($total, 2);
}
```

4.6 Organização em arquivos e conversão para JSON (OO7 e OO8)

Cada classe fica em um arquivo de `src/` e é carregada com `require_once`. As filhas carregam a mãe, e o `index.php` carrega o que usa:

```
src/
  Produto.php  ProdutoUnitario.php  ProdutoPorPeso.php
  ProdutoPerecivel.php  ItemPedido.php  Pedido.php

// public/index.php
require_once __DIR__ . '/../src/ProdutoUnitario.php';
require_once __DIR__ . '/../src/ProdutoPorPeso.php';
require_once __DIR__ . '/../src/Pedido.php';
```

Todas as classes que aparecem no JSON têm `paraArray()`. Esse método é necessário porque `json_encode()` de um objeto com atributos `private` ou `protected` devolve `{}`. A filha reaproveita o array da mãe com `array_merge`:

```
// ProdutoPorPeso
public function paraArray(): array
{
    return array_merge(parent::paraArray(), [
        'peso_minimo' => $this->pesoMinimo,
    ]);
}
```

5 Camada web

Todo o código PHP fica no topo do `public/index.php` e o HTML vem depois (W5). Todo dado mostrado no HTML passa pela função `e()`, que chama `htmlspecialchars` com `ENT_QUOTES` e UTF-8 (W4).

5.1 Rotas

URL	Status	O que mostra
`/`	200	Página inicial
`/?pagina=produtos`	200	Catálogo de produtos
`/?pagina=produto&id=3`	200 ou 404	Detalhe de um produto
`/?pagina=pedidos`	200	Lista de pedidos
`/?pagina=pedido&id=1`	200 ou 404	Itens e total de um pedido
`/?pagina=novo-pedido`	200	Formulário (GET) e criação do pedido (POST)
`/?pagina=xyz`	404	"Página não encontrada."
`/?formato=json`	200 ou 404	Os mesmos dados em JSON

5.2 Páginas e status 404 (W1)

Um `switch` sobre `$pagina` define o status e os dados. Se a página, o produto ou o pedido não existe, o status vira 404 e o `http_response_code()` é chamado antes de qualquer HTML. Saídas do `curl -i`:

```
$ curl -i "http://localhost:8000/?pagina=produto&id=3"
HTTP/1.1 200 OK
Content-type: text/html; charset=UTF-8
...
<title>Carne de sol | Feira Digital do Sertão</title>

$ curl -i "http://localhost:8000/?pagina=produto&id=99"
HTTP/1.1 404 Not Found
Content-type: text/html; charset=UTF-8
...
<title>Não encontrado | Feira Digital do Sertão</title>
<p>Produto não encontrado.</p>

$ curl -i "http://localhost:8000/?pagina=xyz"
HTTP/1.1 404 Not Found
...
<p>Página não encontrada.</p>
```

Na primeira linha de cada resposta aparece o status. O `200` do produto 3 vem do caso em que ele existe, e os dois `404` vêm dos casos em que o produto ou a página não existem. O corpo continua sendo uma página HTML normal, com o mesmo cabeçalho e rodapé.

5.3 Rota JSON (W2)

Com `?formato=json`, o sistema envia o cabeçalho `Content-Type: application/json; charset=utf-8`, devolve os dados de `paraArray()` com `json_encode` e encerra com `exit`, sem enviar HTML. Quando o item não existe, o JSON também vem com status 404.

```
$ curl -i "http://localhost:8000/?pagina=produto&id=3&formato=json"
HTTP/1.1 200 OK
Host: localhost:8000
Content-Type: application/json; charset=utf-8

{
    "id": 3,
    "nome": "Carne de sol",
    "tipo": "Por peso",
    "preco": 48,
```

```
    "peso_minimo": 0.5
}

$ curl -i "http://localhost:8000/?pagina=produto&id=99&formato=json"
HTTP/1.1 404 Not Found
Content-Type: application/json; charset=utf-8

{
    "erro": "Produto não encontrado."
}
```

O cabeçalho `Content-Type` diz ao cliente que o corpo é JSON. O corpo começa direto em `{`, sem `<!DOCTYPE>`, porque o `exit` encerra o script antes do HTML. O campo `peso_minimo` só aparece nos produtos por peso, por causa da sobrescrita de `paraArray()`.

5.4 Formulário com POST (W3)

O formulário da página `novo-pedido` cria um objeto `Pedido` com os dados recebidos. O `index.php` troca a vírgula decimal por ponto, confere com `is_numeric` e só então cria o `ItemPedido`. Se uma regra recusar os dados, o `catch` guarda a mensagem da exceção e ela aparece na página. Com 1,5 unidade de pão de queijo:

```
$ curl -i -X POST -d 'cliente=João&produto[]=1&quantidade[]=1.5' \
    "http://localhost:8000/?pagina=novo-pedido"
HTTP/1.1 200 OK
...
<p class="erro">Produto unitário só aceita quantidade inteira.</p>
```

A Figura 2 mostra a página com essa recusa e a Figura 3 mostra um pedido aceito (3 pães de queijo e 1,5 kg de carne de sol, total de R$ 84,00).

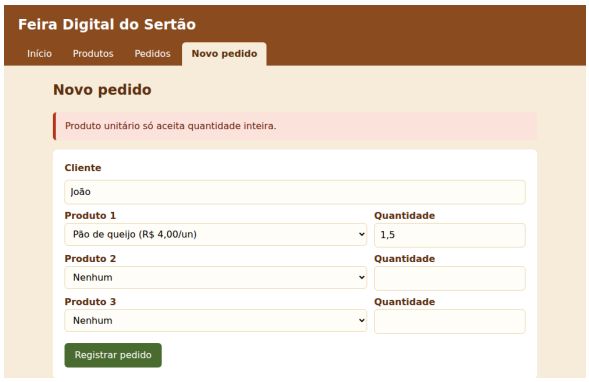


Figura 2: Formulário recusando uma quantidade fracionada



Figura 3: Pedido criado pelo formulário

6 Testes

O arquivo `teste.php` roda no terminal (`php teste.php`) e mostra, em sequência, cada regra funcionando e cada recusa. A saída completa é:

```
=== Teste 1: polimorfismo (mesmo método, respostas diferentes) ===
Pão de queijo [Unitário]: 2 x R$ 4,00 = R$ 8,00
Carne de sol [Por peso]: 2 x R$ 48,00 = R$ 96,00
Iogurte [Perecível]: 2 x R$ 10,00 = R$ 14,00
```

```
Queijo coalho [Perecível]: 2 x R$ 30,00 = R$ 60,00

=== Teste 2: sobrescrita de calcularPreco() no perecível ===
Iogurte (vence em 1 dia), 1 un: R$ 7,00 (esperado R$ 7,00)
Queijo (vence em 10 dias), 1 un: R$ 30,00 (esperado R$ 30,00)

=== Teste 3: composição (pedido com itens e total) ===
Pão de queijo x3 = R$ 12,00
Carne de sol x1.5 = R$ 72,00
Iogurte x2 = R$ 14,00
Total de Maria: R$ 98,00 (esperado R$ 98,00)

=== Teste 4: regras de Produto (nome, preço e quantidade) ===
OK, recusado: O nome do produto não pode ficar vazio.
OK, recusado: O preço do produto deve ser maior que zero.
OK, recusado: A quantidade deve ser maior que zero.

=== Teste 5: regras das filhas (unitário e por peso) ===
OK, recusado: Produto unitário só aceita quantidade inteira.
OK, recusado: A compra mínima de Carne de sol é 0,5 kg.
OK, recusado: O peso mínimo deve ser maior que zero.

=== Teste 6: regras do produto perecível (validade) ===
OK, recusado: A validade deve ser uma data válida no formato AAAA-MM-DD.
OK, recusado: O produto perecível não pode estar vencido.

=== Teste 7: regras do pedido (cliente e produto repetido) ===
OK, recusado: O nome do cliente não pode ficar vazio.
OK, recusado: O produto Pão de queijo já está no pedido.

=== Teste 8: paraArray() e JSON do pedido ===
{
    "cliente": "Maria",
    "itens": [
        {
            "produto": {
                "nome": "Pão de queijo",
                "tipo": "Unitário",
                "preco": 4
            },
            "quantidade": 3,
            "subtotal": 12
        },
        {
            "produto": {
                "nome": "Carne de sol",
                "tipo": "Por peso",
                "preco": 48,
                "peso_minimo": 0.5
            },
            "quantidade": 1.5,
            "subtotal": 72
        },
        {
            "produto": {
                "nome": "Iogurte",
                "tipo": "Perecível",
                "preco": 10,
                "validade": "2026-11-04",
                "em_promocao": true
            },
            "quantidade": 2,
            "subtotal": 14
        }
    ],
    "total": 98
}
```

O que cada teste comprova:

Teste	O que comprova
1	Polimorfismo: o mesmo `getTipo()` e `calcularPreco()` em 4 objetos de 3 classes, cada um respondendo do seu jeito. O iogurte, que vence amanhã, sai por R$ 14,00 em vez de R$ 20,00.
2	Sobrescrita de `calcularPreco()`: 30% de desconto para quem vence em 1 dia e preço cheio para quem vence em 10 dias.
3	Composição: o pedido soma os subtotais dos três itens (12 + 72 + 14 = 98).
4	Regras da mãe: nome vazio, preço zero e quantidade zero são recusados.
5	Regras das filhas: quantidade fracionada no unitário, compra abaixo do mínimo e peso mínimo zero.
6	Regras do perecível: validade que não é data e produto vencido.
7	Regras do pedido: cliente vazio e produto repetido.
8	`paraArray()` e JSON: o pedido inteiro, com os itens e os produtos dentro, virou JSON. O iogurte mostra `em_promocao: true`.

Em todos os testes de recusa, a linha `OK, recusado` mostra a mensagem da própria exceção. Se uma regra deixasse passar o dado inválido, a linha `ERRO` apareceria no lugar.

7 Ferramentas utilizadas

Ferramenta	Para que foi usada
PHP 8.3	Linguagem do sistema e servidor embutido (`php -S`) para rodar o site
Neovim	Editor para escrever o código e a documentação
Git e GitHub	Versionamento, issues, branches, pull requests e tags dos marcos
curl	Testar as rotas no terminal (`curl -i`) e conferir status e cabeçalhos
Navegador e DevTools	Ver as páginas e investigar erros no console
LLM: Claude	Explicar conceitos e erros, revisar o código e ajudar nos testes e na documentação
Gravador de tela: OBS Studio	Gravar o vídeo de apresentação

8 Uso de LLM

Usei o Claude como ferramenta de aprendizado, e o diário completo está em `docs/llm.md`. Registrei 5 interações:

1. **`protected` e `private`.** Pedi a diferença usando o meu código. Aprendi que `nome` e `preco` são `protected` porque as filhas precisam deles, e `pesoMinimo` é `private` porque é uma regra só da `ProdutoPorPeso`.

2. **CSS que não carregava.** O navegador recusava o `estilo.css` por causa do tipo MIME. Pedi as causas prováveis e como verificar cada uma, sem a correção pronta. Descobri que, com `-t public`, a pasta `public` já é a raiz do site, então o arquivo é `/estilo.css`, e não `/public/estilo.css`.

3. **Revisão da classe `Pedido`.** Pedi para revisar só contra os requisitos. A resposta disse que todos estavam atendidos e apontou dois cuidados: o `strcasecmp` não ignora diferença de maiúsculas e minúsculas em letras com acento, e um pedido sem itens tem total zero.

4. **Justificativa da herança.** Pedi uma revisão do texto da seção de herança da modelagem.

5. **`strict_types` e `$_POST`.** Aprendi que tudo que vem do `$_POST` é texto, que com `strict_types=1` passar texto a um parâmetro `float` lança `TypeError`, e que esse erro não é uma `Exception`, então o `catch (Exception)` não o pega. Também aprendi a cuidar da vírgula decimal.

O que apliquei no código. O `index.php` converte a vírgula em ponto e confere com `is_numeric` antes de criar o `ItemPedido`, o que evita o `TypeError` da interação 5. O formulário recusa pedido sem nenhum produto, e o caminho do CSS é `/estilo.css`.

Limitação que continua. O `Pedido` ainda usa `strcasecmp`. Testei, e `Maçã` e `MAÇÃ` são tratados como produtos diferentes, enquanto `Maca` e `MACA` são iguais. Comparar com `mb_strtolower` nos dois lados resolveria. Nenhum produto do catálogo tem esse problema, mas ele existe.

Reflexão. A LLM ajudou mais quando eu pedi explicação e verificação, e não código pronto: nas interações 2 e 5 a resposta foi o motivo do problema e um jeito de testar, e a correção ficou no meu código. O risco de usar a LLM seria aceitar um código que eu não consigo explicar, e por isso escrevi nos prompts que não queria o código reescrito. Também aprendi que a resposta da LLM precisa ser conferida no terminal, como fiz com o `strcasecmp`.

9 Dificuldades, aprendizados e conclusão

Dificuldades. A primeira foi o CSS que não carregava, por não entender que o `-t public` muda a raiz do site. A segunda foi o formulário: os dados do `$_POST` chegam como texto, e com `strict_types=1` isso gera `TypeError`, que escapa do `catch (Exception)`. A terceira foi decidir onde cada regra deve ficar. Cheguei a esta divisão: o que vale para todo produto fica em `Produto`, e o que depende do tipo fica na filha, como a quantidade inteira e o peso mínimo.

Aprendizados. Aprendi que o construtor com validação impede que um objeto inválido exista, e que a sobrescrita com `parent::` reaproveita a regra da mãe e acrescenta a da filha. Aprendi também que `json_encode` de um objeto com atributos `private` devolve `{}`, por isso o `paraArray()`. Por fim, que o `teste.php` mostra cada regra funcionando e recusando, e serve de roteiro para conferir o sistema.

O que faria diferente. Trocaria o `strcasecmp` por `mb_strtolower`, e faria a persistência em arquivo JSON (o bônus), para que o pedido criado pelo formulário continuasse lá na próxima requisição. Hoje o pedido criado só aparece na resposta do POST, porque os objetos vivem só durante uma requisição.

Conclusão. O sistema cumpre os requisitos de orientação a objetos (OO1 a OO8) e da camada web (W1 a W5): 6 classes, uma hierarquia com 3 filhas, polimorfismo, composição, 10 regras recusadas com exceção e tratadas com `try/catch`, páginas com 404, rota JSON e formulário com recusa.

Links

- Repositório: https://github.com/221halef/pweb1-trabalho-oo
- Vídeo: o link está no início do `README.md` do repositório.